

Document number: P2643R1

Date: 2023-05-18

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>

Authors: Gonzalo Brito Gadeschi, Olivier Giroux, Thomas Rodgers

Audience: Concurrency

Improving C++ concurrency features

Revisions

r0 - (Kona Drafts)

- Added discussion of pros/cons of returning `pair<T, bool>` vs. `optional<T>`.
- Added discussion of timed waits and freestanding, given that `<chrono>` is not part of freestanding.
- Added wording for `barrier::try_wait_for` and `barrier::try_wait_until`.
- Added motivating examples.
- Fixed `barrier::try_wait_for/_until` signatures; they were incorrectly accepting an `arrival_token&&`, but since these can be called in a loop consuming the `arrival_token` is incorrect.
- Removed discussion of 'hinted' wait mechanism. The design surface area of this proposal is such that it should be a separate paper, brought forward an interested party.
- Removed fallible untimed `try_wait`.

r1 - (Varna Draft)

- Removed timed waiting for freestanding, to reflect Kona guidance, and added discussion.
- Removed pros/cons discussion of returning `optional<T>` VS `pair<bool, T>` VS `T`, reflecting Kona guidance.
- Re-added fallible untimed `try_wait` with rationale, reflecting Kona guidance.

Introduction

P1135R6 (<https://wg21.link/p1135r6>) introduced several new concurrency primitives to the C++20 concurrency library:

- `<atomic>` : added the class `atomic_flag`, the `wait` and `notify_one/_all` to class template `atomic<>`, and free function versions of these.
- `<semaphore>` : added class template `counting_semaphore<>` and class `binary_semaphore`.
- `<barrier>`, `<latch>` : added class template `barrier<>` and class `latch`.

Though each element included was long coming, and had much implementation experience behind it, fresh user feedback tells us that some improvements could still be made:

1. Return last observed value from `atomic::wait`; this value is lost otherwise.
2. Add timed versions of `atomic::wait` and other concurrency primitives like `barrier` and `latch`, to make it easier to implement concurrency primitives on top of `atomic` that expose timed waiting facilities themselves, e.g., like `<semaphore>`.
3. Avoid spurious polling in `atomic::wait` by accepting a predicate.

This proposal proposes extensions to address these shortcomings. This branch (<https://gitlab.com/rodgert/gcc/-/commit/dab70764cd68236472656a9394a32185a9ccc890>) demonstrates its implementability in `libstdc++`.

Design

The design of the features above is mostly orthogonal, and this section explores them independently.

Return last observed value on success

Desing: `::wait` APIs change the return type signature from returning ~~`void`~~ `T` `wait(...)`;

Example 0:

Before	After
<pre>atomic<int> a(42); a.wait(42); auto o = a.load(); assert(o != 42); // MAY FAIL!</pre>	<pre>atomic<int> a(42); auto o = a.wait(42); assert(o != 42); // OK!</pre>

The `atomic<T>::wait` method guarantees that the thread is unblocked only if the value changed.

Before this paper, the new `atomic<T>` value is not returned to the caller. This has the following two shortcomings:

- **That value is lost forever:** after the thread is unblocked, the value might change before the unblocked thread calls `atomic<T>::load` (ABA Problem (https://en.wikipedia.org/wiki/ABA_problem)).
- **Applications need the new value often:** which forces them to call `atomic<T>::load`, even though `atomic::wait<T>` most likely already loaded the value, to test that it did change.

After this paper, the value returned by `wait` is returned to the caller, eliminating the need for the subsequent load.

This is an ABI breaking change that could be resolved by picking a different to-be-determined name for these APIs.

Fallible and timed versions of wait APIs

The design of the fallible timed versions of wait APIs adds three new APIs. For `atomic` these are

```
template <class T>
optional<T> atomic<T>::try_wait(T, memory_order);

template <class T, class Rep, class Period>
optional<T> atomic<T>::try_wait_for(T, duration<Rep, Period> const&, memory_order);

template <class T, class Clock, class Duration>
optional<T> atomic<T>::try_wait_until(T, time_point<Clock, Duration> const&, memory_order);
```

They are non-blocking, i.e., they eventually return to the caller in a finite-set of steps, even if the value did not change. This enables the application to “do something else” before attempting to wait again.

On failure, i.e., if the value did not change, they return `nullopt` and the operation has no effects (it does not synchronize). On success, they return an `optional<T>` containing the last observed value, which is guaranteed to be different from the one the call site waited on.

The untimed `try_wait` overload attempts to wait for a finite *unspecified* duration. The implementation may pick a different duration every time, enabling it decide for how long the operation should attempt to wait based on dynamic information, like the load of the system.

Since `<chrono>` and `<optional>` are not freestanding, these APIs will not be available in freestanding implementations. We could attempt to improve this by:

- Supporting the subset of `optional` APIs that do not throw exceptions in freestanding.

- Supporting the `<chrono>` durations in freestanding.

Example 1: the atomic variable `t` tracks how many tasks need to still be processed, and the application prints every second - if it is still waiting - how many tasks remain:

Before	After
<pre>atomic<int> t; int old = 1; auto b = clock::now(); while (old != 0) { old = t.load(); if (auto e = clock::now(); (e - b) > 1s) { cout << old; b = e; } }</pre>	<pre>atomic<int> t; auto old = 1; while (old != 0) { auto o = t.try_wait_for(old, 1s); old = o.has_value()? o.value() : old; cout << old; }</pre>

Before this proposal, applications need to re-implement `atomic<T>::wait` logic, and like in this example, may accidentally call `atomic<T>::load` in a loop without any back-off.

After this proposal, `try_wait_for` expresses the application's intent to wait for a certain duration of time, enabling the implementation to wait efficiently.

For `barrier` and `latch` they look like:

```
template <class CF>
bool barrier<CF>::try_wait(arrival_token& tok) const;
template <class CF, class Rep, class Period>
bool barrier<CF>::try_wait_for(arrival_token& tok, duration<Rep, Period> const& rel_time) const;
template <class CF, class Clock, class Duration>
bool barrier<CF>::try_wait_until(arrival_token& tok, time_point<Clock, Duration> const& abs_time) const;

// Available since C++20
// bool latch::try_wait() const noexcept;
template <class Rep, class Period>
bool latch::try_wait_for(duration<Rep, Period> const& rel_time) const;
template <class Clock, class Duration>
bool latch::try_wait_until(time_point<Clock, Duration> const& abs_time) const;
```

Example 2: using a `barrier` which tracks the completion of `task_count` tasks:

```

std::barrier b(task_count);

auto n = process_thread_local_tasks();
auto t = b.arrive(n);

while (!b.try_wait_for(t, 1ms)) {
    auto s = steal_and_process_tasks();
    b.arrive(s);
}

```

Predicate versions

There is no wording for the predicate versions proposed in this revision of this paper.

When the program is waiting for a condition different from “not equal to”, there is an added re-try loop around the `wait` operation in the program. This loop causes *each* call to `wait` to be performed as if it were the *first* call to `wait`, oblivious to the fact that the program has already been waiting for some time. This leads to re-executing the short-term polling strategy.

Taking a predicate instead of a value allows us to push the program-defined condition inside of `atomic::wait`, delete the outer loop, and allows the implementation to track time spent.

At least two implementations currently implement `atomic::wait` in terms of a wait taking a predicate.

The authors see two APIs that are both equivalent in terms of implementation complexity and usability, but have an opposite API depending on how the semantics of the predicate are defined.

Both options have this signature `T atomic<T>::wait_with_predicate(Predicate&& p)` and the predicate takes a `T` and returns a `bool`, but the predicate returns:

- Option 1: `true` if we synchronizes
 - Understood as the wait succeeds
- Option 2: `false` if we synchronizes
 - Understood as “the value changed” (value compared not-equal - like `atomic::wait(T)` waits until `T` changes)

Feedback needed: We should pick an option before working on wording.

Example 3: same example as Example 1 using Option 1 above (`true` if we synchronize):

Before	After
<pre> atomic<int> t; int old = 1; auto b = clock::now(); while (old != 0) { old = t.load(); auto e = clock::now(); if ((e - b) > 1s) { cout << old; b = e; } } </pre>	<pre> atomic<int> t; int old = t.load(); auto p = [&](int v) { old = v; return v == 0; }; while(!t.try_wait_with_predicate_for(p, 1s).has_value()) { cout << old; } </pre>

In Example 1, the “After this paper” implementation has the issue that `try_wait_for(old, 1s)` returns as soon as `old` changes. That is, while the goal of the application is to wait for the value becoming zero, and it could wait up to 1s for that, if after 0.1s the value changes from 10 to 9 this API will return.

The “After this paper” implementation in Example 2 fixes that using the fallible timed `_with_predicate` version. The predicate is called every time the value changes, but it doesn’t “succeed” until the value is 0.

Wording

Return last observed value from `atomic::wait`

To [atomics.ref.generic.general] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        void wait(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

UNRESOLVED QUESTION: all `atomic_ref` types are missing `volatile wait` overloads. Should that be fixed as part of this work?

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```
voidT wait(T old, memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions*: order is neither memory_order::release nor memory_order::acq_rel .
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates load(order) and compares its value representation for equality against that of old .
 2. If they compare unequal, returns the result of the evaluation of load(order) in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: This function is an atomic waiting operation (atomics.wait (http://eel.is/c++draft/atomics#wait)) on atomic object *ptr .

To [atomics.ref.int] (http://eel.is/c++draft/atomics.ref.int):

```
namespace std {  
    template<> struct atomic_ref<integral> {  
        voidT wait(integral, memory_order = memory_order::seq_cst) const noexcept;  
    };  
}
```

To [atomics.ref.float] (http://eel.is/c++draft/atomics.ref.float):

```
namespace std {  
    template<> struct atomic_ref<floating-point> {  
        voidT wait(floating-point, memory_order = memory_order::seq_cst) const noexcept;  
    };  
}
```



To [atomics.ref.pointer] (http://eel.is/c++draft/atomics.ref.pointer):

```
namespace std {  
    template<class T> struct atomic_ref<T*> {  
        voidT* wait(T*, memory_order = memory_order::seq_cst) const noexcept;  
    };  
}
```

To [atomics.types.generic.general] (http://eel.is/c++draft/atomics.types.generic.general):

```

namespace std {
    template<class T> struct atomic {
        voidT wait(T, memory_order = memory_order::seq_cst) const volatile noexcept;
        voidT wait(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```

voidT wait(T old, memory_order order = memory_order::seq_cst) const volatile noexc
voidT wait(T old, memory_order order = memory_order::seq_cst) const noexcept;

```

- *Preconditions*: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)).

To [atomics.types.int] (<http://eel.is/c++draft/atomics.types.int>):

```

namespace std {
    template<> struct atomic<integral> {
        voidT wait(integral, memory_order = memory_order::seq_cst) const volatile noex
        voidT wait(integral, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.types.float] (<http://eel.is/c++draft/atomics.types.float>):


```

namespace std {
    template<> struct atomic<floating-point> {
        voidT wait(floating-point, memory_order = memory_order::seq_cst) const volatile
        voidT wait(floating-point, memory_order = memory_order::seq_cst) const noexcept
    };
}

```

To [atomics.types.pointer] (<http://eel.is/c++draft/atomics.types.pointer>):

```

namespace std {
    template<class T> struct atomic<T*> {
        voidT* wait(T*, memory_order = memory_order::seq_cst) const volatile noexcept;
        voidT* wait(T*, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [util.smartptr.atomic.shared] (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```

namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        voidshared_ptr<T> wait(shared_ptr<T> old, memory_order = memory_order::seq_cst
    };
}

```

and

```

voidshared_ptr<T> wait(shared_ptr<T> old, memory_order order = memory_order::seq_c

```

- *Preconditions*: order is neither memory_order::release nor memory_order::acq_rel.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates load(order) and compares it to old.
 2. If the two are not equivalent, returns the result of the evaluation of load(order) in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: Two shared_ptr objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation (atomics.wait (<http://eel.is/c++draft/atomics.wait>)).

To [util.smartptr.atomic.weak] (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        voidweak_ptr<T> wait(weak_ptr<T> old, memory_order = memory_order::seq_cst) co
    };
}
```

```
voidweak_ptr<T> wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst)
```

- *Preconditions*: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)).

No changes to [atomics.nonmembers] (<http://eel.is/c++draft/atomics.nonmembers>) are needed.

No changes to [atomic.flag] (<http://eel.is/c++draft/atomics.flag>)'s `wait` APIs are needed.

Fallible and timed-versions of `::wait` APIs

To [atomics.ref.generic.general] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        optional<T> try_wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T> try_wait_for(
            T, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T> try_wait_until(
            T, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```

optional<T> try_wait(memory_order = memory_order::seq_cst) const noexcept;
template <class Rep, class Period>
optional<T> try_wait_for(T old,
    chrono::duration<Rep, Period> const& rel_time,
    memory_order order = memory_order::seq_cst
) const noexcept;
template <class Clock, class Duration>
optional<T> try_wait_until(T old,
    chrono::time_point<Clock, Duration> const& abs_time,
    memory_order order = memory_order::seq_cst
) const noexcept;

```

- Preconditions: order is neither memory_order::release nor memory_order::acq_rel.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates load(order) and compares its value representation for equality against that of old.
 2. If they compare unequal, returns the result of the evaluation of load(order) in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously or the timeout expired. If it is unblocked by the timeout there is no effect and it returns nullopt.

The timeout expires (thread.req.timing (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after abs_time (for try_wait_until) or when at least rel_time has passed from the start of the function (for try_wait_for).

An implementation should ensure that `try_wait_for` and `try_wait_until` do not consistently return `nullopt` in the absence of contending atomic operations.

The timeout for `try_wait` is finite but otherwise unspecified.

- *Throws:* Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To `[atomics.ref.int]` (<http://eel.is/c++draft/atomics.ref.int>):

```
namespace std {  
    template<> struct atomic_ref<integral> {  
        optional<integral> try_wait(memory_order = memory_order::seq_cst) const noexcept  
        template <class Rep, class Period>  
        optional<integral> try_wait_for(  
            integral, chrono::duration<Rep, Period> const& rel time,  
            memory_order = memory_order::seq_cst  
        ) const noexcept;  
        template <class Clock, class Duration>  
        optional<integral> try_wait_until(  
            integral, chrono::time_point<Clock, Duration> const& abs time,  
            memory_order = memory_order::seq_cst  
        ) const noexcept;  
    };  
}
```



To `[atomics.ref.float]` (<http://eel.is/c++draft/atomics.ref.float>):

```

namespace std {
    template<> struct atomic_ref<floating-point> {
        optional<floating-point> try wait(memory_order = memory_order::seq_cst) const
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.ref.pointer] (<http://eel.is/c++draft/atomics.ref.pointer>):

```

namespace std {
    template<class T> struct atomic_ref<T*> {
        optional<T*> try wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T*> try wait for(
            T*, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T*> try wait until(
            T*, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.types.generic.general] (<http://eel.is/c++draft/atomics.types.generic.general>):

```

namespace std {
    template<class T> struct atomic {
        optional<T> try_wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T> try_wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        optional<T> try_wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<T> try_wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T> try_wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```

optional<T> try_wait(memory_order = memory_order::seq_cst) const noexcept;
template <class Rep, class Period>
optional<T> try_wait_for(T old,
                        chrono::duration<Rep, Period> const& rel_time,
                        memory_order order = memory_order::seq_cst
                        ) const noexcept;
template <class Rep, class Period>
optional<T> try_wait_for(T old,
                        chrono::duration<Rep, Period> const& rel_time,
                        memory_order order = memory_order::seq_cst
                        ) const volatile noexcept;
template <class Clock, class Duration>
optional<T> try_wait_until(T old,
                        chrono::time_point<Clock, Duration> const& abs_time,
                        memory_order order = memory_order::seq_cst
                        ) const noexcept;
template <class Clock, class Duration>
optional<T> try_wait_until(T old,
                        chrono::time_point<Clock, Duration> const& abs_time,
                        memory_order order = memory_order::seq_cst
                        ) const volatile noexcept;

```

EDITORIAL: analogous to `atomic_ref`. Intentionally left out from the current revision of this paper.

To [atomics.types.int] (<http://eel.is/c++draft/atomics.types.int>):

```

namespace std {
    template<> struct atomic<integral> {
        optional<integral> try wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<integral> try wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<integral> try wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<integral> try wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<integral> try wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.float] (<http://eel.is/c++draft/atomics.types.float>):


```

namespace std {
    template<> struct atomic<floating-point> {
        optional<floating-point> try wait(memory_order = memory_order::seq_cst) const
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.pointer] (<http://eel.is/c++draft/atomics.types.pointer>):

```

namespace std {
    template<class T> struct atomic<T*> {
        optional<T*> try_wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T*> try_wait for(
            T*, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<T*> try_wait for(
            T*, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<T*> try_wait until(
            T*, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T*> try_wait until(
            T*, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [util.smartptr.atomic.shared] (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```

namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        optional<shared_ptr<T>> try_wait(memory_order = memory_order::seq_cst) const
        template <class Rep, class Period>
        optional<shared_ptr<T>> try_wait for(
            shared_ptr<T>l, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<shared_ptr<T>> try_wait until(
            shared_ptr<T>, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

EDITORIAL: analogous to the `try_wait_for / _until` APIS of `atomic_ref`, with `shared_ptr / weak_ptr` tweaks. Intentionally left out of the current revision of this paper.

To `[util.smartptr.atomic.weak]` (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        optional<weak_ptr<T>> try_wait(memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<weak_ptr<T>> try_wait_for(
            weak_ptr<T> l, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<weak_ptr<T>> try_wait_until(
            weak_ptr<T>, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}
```

EDITORIAL: analogous to the `try_wait_for / _until` APIS of `atomic_ref`, with `shared_ptr / weak_ptr` tweaks. Intentionally left out of the current revision of this paper.

EDITORIAL: No changes to `[atomics.nonmembers]` (<http://eel.is/c++draft/atomics.nonmembers>) are needed.

To `[atomic.flag]` (<http://eel.is/c++draft/atomics.flag>):

```

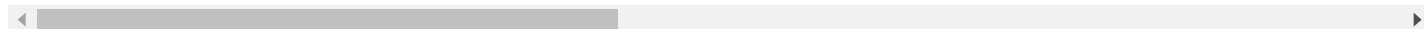
namespace std {
    struct atomic_flag {
        bool try_wait(memory_order = memory_order::seq_cst) const noexcept;
        bool try_wait(memory_order = memory_order::seq_cst) const noexcept volatile;
        template <class Rep, class Period>
        bool try_wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        bool try_wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        bool try_wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        bool try_wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

bool atomic_flag::try_wait(const atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait(const volatile atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait_explicit(const atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait_explicit(const volatile atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) const;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) volatile;

bool atomic_flag::try_wait_for(const atomic_flag* object, bool old, chrono::duration<Rep, Period> const& rel_time) const;
bool atomic_flag::try_wait_for(const volatile atomic_flag* object, bool old, chrono::duration<Rep, Period> const& rel_time) volatile;
bool atomic_flag::try_wait_for_explicit(const atomic_flag* object, bool old, chrono::duration<Rep, Period> const& rel_time) const;
bool atomic_flag::try_wait_for_explicit(const volatile atomic_flag* object, bool old, chrono::duration<Rep, Period> const& rel_time) volatile;
bool atomic_flag::try_wait_for(bool old, chrono::duration<Rep, Period> const& rel_time, memory_order order = memory_order::seq_cst) const;
bool atomic_flag::try_wait_for(bool old, chrono::duration<Rep, Period> const& rel_time, memory_order order = memory_order::seq_cst) volatile;

bool atomic_flag::try_wait_until(const atomic_flag* object, bool old, chrono::time_point<Clock, Duration> const& time_point) const;
bool atomic_flag::try_wait_until(const volatile atomic_flag* object, bool old, chrono::time_point<Clock, Duration> const& time_point) volatile;
bool atomic_flag::try_wait_until_explicit(const atomic_flag* object, bool old, chrono::time_point<Clock, Duration> const& time_point) const;
bool atomic_flag::try_wait_until_explicit(const volatile atomic_flag* object, bool old, chrono::time_point<Clock, Duration> const& time_point) volatile;
bool atomic_flag::try_wait_until(bool old, chrono::time_point<Clock, Duration> const& time_point, memory_order order = memory_order::seq_cst) const;
bool atomic_flag::try_wait_until(bool old, chrono::time_point<Clock, Duration> const& time_point, memory_order order = memory_order::seq_cst) volatile;



For `atomic_flag::try_wait`, `atomic_flag::try_wait_for`, and `atomic_flag::try_wait_until`, let `order` be `memory_order::seq_cst`. Let `flag` be object for the non-member functions, and `this` for the member functions.

- *Preconditions:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects:* Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `try_wait_until`) or when at least `rel_time` has passed from the start of the function (for `try_wait_for`).

An implementation should ensure that `try_wait_for` and `try_wait_until` do not consistently return `nullopt` in the absence of contending atomic operations.

The timeout for `try_wait` is finite but otherwise unspecified.

- *Throws:* Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To [`atomics.wait`] (<http://eel.is/c++draft/atomics.wait>):

2. [Note 2: The following functions are atomic waiting operations:

- 2.1 `atomic<T>::wait`, `atomic<T>::wait_for`, `atomic::<T>::wait_until`,
 - 2.2 `atomic_flag::wait`, `atomic_flag::wait_for`, `atomic_flag::wait_until`,
 - 2.3 `atomic_wait` ~~and~~, `atomic_wait_explicit`, `atomic_wait_for`,
`atomic_wait_for_explicit`, `atomic_wait_until`, `atomic_wait_until_explicit`,
 - 2.4 `atomic_flag_wait` ~~and~~, `atomic_flag_wait_explicit`, `atomic_flag_wait_for`,
`atomic_flag_wait_for_explicit`, `atomic_flag_wait_until`,
`atomic_flag_wait_until_explicit`, ~~and~~
 - 2.5 `atomic_ref<T>::wait`, `atomic_ref<T>::wait_for`, `atomic_ref<T>::wait_until`.
- end note]

UNRESOLVED QUESTION: do we need to change something else for the non-member versions of `try_wait` for and `try_wait_until` operations?

To [`thread.barrier`] (<http://eel.is/c++draft/thread.barrier>):

```
namespace std {
    template <class Completion Function>
    class barrier {

    public:
        bool try_wait(arrival token& tok) const;
        template <class Rep, class Period>
        bool try_wait_for(arrival token& tok, chrono::duration<Rep, Period> const& rel
        template <class Clock, class Duration>
        bool try_wait_until(arrival token& tok, chrono::time_point<Clock, Duration> co
    };
}
```

UNRESOLVED QUESTION: `arrival_token&` vs `arrival_token const&` ?

Note: `try_wait_for` / `try_wait_until` do not take ownership of the `arrival_token` because if they fail, the user still needs the token to be able to call them again.

```

bool try_wait(arrival token& tok) const;
template <class Rep, class Period>
bool try_wait_for(arrival token& tok, chrono::duration<Rep, Period> const& rel tim
template <class Clock, class Duration>
bool try_wait_until(arrival token& tok, chrono::time_point<Clock, Duration> const&

```

- Preconditions: arrival is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.
- Effects: Blocks at the synchronization point associated with arrival until the phase completion step of the synchronization point's phase is run or the timeout expired. If it is unblocked by the timeout there is no effect and it returns false; otherwise, it returns true.

The timeout expires (thread.req.timing (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after abs_time (for try_wait_until) or when at least rel_time has passed from the start of the function (for try_wait_for).

The timeout for try_wait is finite but otherwise unspecified.

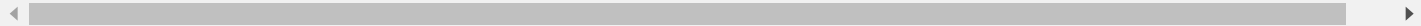
An implementation must ensure that try_wait_for and try_wait_until do not consistently return false after the phase completion step associated with arrival has run.

[Note: If arrival is associated with the synchronization point for a previous phase, the call returns immediately. — end note]

- Throws: system_error when an exception is required (thread.req.exception (<http://eel.is/c++draft/thread.req.exception>)) or timeout-related exceptions (thread.req.timing (<http://eel.is/c++draft/thread.req.timing>)).
- Error conditions: Error conditions: Any of the error conditions allowed for mutex types (thread.mutex.requirements.mutex (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).

To thread.latch (<http://eel.is/c++draft/thread.latch>):

```
namespace std {  
    class latch {  
    public:  
        bool try wait(.) const;  
        template <class Rep, class Period>  
        bool try wait for(chrono::duration<Rep, Period> const& rel time) const;  
        template <class Clock, class Duration>  
        bool try wait until(chrono::time_point<Clock, Duration> const& abs time) const  
    };  
}
```



```
bool try wait(.) const;  
template <class Rep, class Period>  
bool try wait for(chrono::duration<Rep, Period> const& rel time) const;  
template <class Clock, class Duration>  
bool try wait until(chrono::time_point<Clock, Duration> const& abs time) const;
```

EDITORIAL: semantics intentionally omitted from the current revision of this paper but analogous to `barrier` .